



University of the Philippines

Single-Source Shortest Paths: From Dijkstra to DMMSY

A CMSC 142 portfolio study comparing five SSSP algorithms in pure-Python stdlib, with DMMSY (2025) as the centerpiece.

Zildjian E. California¹ Rey Marvin C. Rizal¹

¹ Department of Mathematics, Physics, and Computer Science,
College of Science and Mathematics, University of the Philippines - Mindanao

CMSC 142 – Design and Analysis of Algorithms · May 22, 2026

- 1 Problem and Motivation
- 2 Baseline Algorithms
- 3 DMMSY (Centerpiece)
- 4 Experimental Evaluation
- 5 Reflection and Author Contributions

Problem and Motivation

Problem

Given a weighted graph $G = (V, E)$ with edge weights $w: E \rightarrow \mathbb{R}$ and a source vertex $s \in V$, compute:

$$d(v) = \min_{\pi: s \rightarrow v} \sum_{e \in \pi} w(e) \quad \text{for every } v \in V.$$

- Edge orientation: directed for Dijkstra, Bellman–Ford, A^* , DMMSY; undirected for Thorup.
- Weight class: non-negative reals for Dijkstra / A^* / DMMSY; signed reals for Bellman–Ford; non-negative integers for Thorup.
- Output: a distance map $d: V \rightarrow \mathbb{R}$ (and, optionally, a predecessor map for path reconstruction).

- **Dijkstra (1959) [1]** – the foundational non-negative-weight algorithm; binary-heap relaxation.
- **Bellman–Ford (1958) [2]**, drawing on Ford (1956) [3] – handles negative weights; detects negative cycles.
- **A* (1968) [4]** – heuristic-guided search; degenerates to Dijkstra at $h \equiv 0$.
- **Thorup (1999) [5]** – linear-time undirected SSSP for non-negative integer weights in the word-RAM model.
- **DMMSY (2025) [6]** – deterministic $O(m \log^{2/3} n)$ directed SSSP; **the centerpiece of this study.**

We re-implement all five in pure-stdlib CPython 3.11+ and benchmark each against Dijkstra on a documented suite.

Baseline Algorithms

Problem Class

Non-negative weighted directed (or undirected) graphs; single source s .

Design Idea

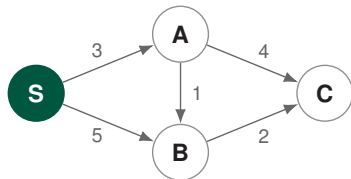
Maintain a frontier set B in a min-heap keyed by tentative distance; repeatedly extract the closest unsettled vertex, settle it, and relax its outgoing edges.

Complexity

Time $O((n+m) \log n)$; space $O(n)$.

Implementation

`src/sssp/dijkstra.py`;
heapq-backed;
7 tests, 100% line coverage.
Source paper: [1].



Step 1: pop S (d=0)

settled	{S:0}
heap	[(3,A),(5,B)]
relaxed	S→A, S→B

Step 3: pop B (d=4)

settled	{S:0, A:3, B:4}
heap	[(5,B*), (6,C), (7,C)]
relaxed	B→C (7→6)

Step 2: pop A (d=3)

settled	{S:0, A:3}
heap	[(4,B), (5,B), (7,C)]
relaxed	A→B (5→4), A→C

Step 4: pop C (d=6) – done

settled	{S:0, A:3, B:4, C:6}
heap	stale entries skipped
result	all distances final

■ settled ■ frontier B^* = stale entry, ignored on pop

Problem Class

Directed graphs with possibly negative edge weights; detect negative-weight cycles reachable from s .

Design Idea

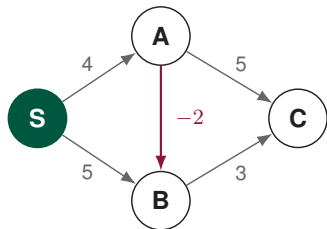
Repeat full-graph edge relaxation $n - 1$ times; one extra pass diagnoses any reachable negative cycle.

Complexity

Time $O(nm)$; space $O(n)$.

Implementation

```
src/sssp/bellman_ford.py;  
reports  
NegativeCycleReport;  
100% line coverage.  
Source papers: [2], [3].
```



Edge scan order

1. (S,A,+4)
2. (S,B,+5)
3. (A,B,-2) ← key relaxation →
4. (B,C,+3)
5. (A,C,+5)

Distance after each round

vertex	init	R1	R2
S	0	0	0
A	∞	4	4
B	∞	5 → 2	2
C	∞	5	5

- Each round sweeps every edge once; relaxing $(A, B, -2)$ after $(S, B, 5)$ lowers $d[B]$ from 5 to 2.
- R2 produces no update $\Rightarrow$ early-exit; the final cycle-check pass also relaxes nothing $\Rightarrow$ `has_cycle = False`.

Problem Class

Goal-directed shortest path with a domain-specific heuristic $h(v)$ lower-bounding $d(v, \text{goal})$.

Design Idea

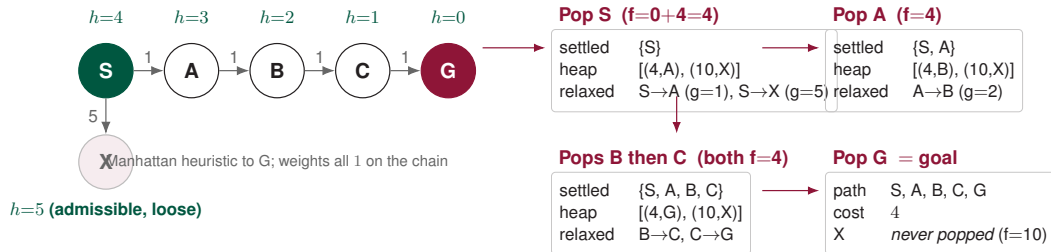
Order the frontier by $f(v) = g(v) + h(v)$ instead of $g(v)$ alone; admissible h preserves optimality.

Complexity

Time $O((n + m) \log n)$ worst case; expands fewer nodes than Dijkstra when h is informative.

Implementation

```
src/sssp/astar.py,  
src/sssp/heuristics.py;  
ships    manhattan,    zero;  
 $A^*(h \equiv 0) \equiv \text{Dijkstra}$  tested.  
Source paper: [4].
```



- Every chain vertex has $f = g + h = 4$, so A* pops them in order and **never expands the dead-end branch X** (its $f = 5 + 5 = 10$).
- Set $h \equiv 0$ and A* degenerates to Dijkstra – the equivalence is asserted as a correctness test in `tests/test_astar.py`.

Problem Class

Undirected graphs with non-negative integer edge weights; word-RAM model.

Design Idea

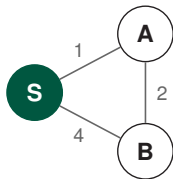
Hierarchical bucket structure exploiting integer weight bits and constant-time word operations to avoid the sorting bottleneck of comparison-based SSSP.

Complexity

Time $O(n + m)$ in the word-RAM model.

Implementation

`src/sssp/thorup99.py`;
11 tests, 99% line coverage.
Python $\neq$ *word-RAM*: caveat documented in the project notes.
Source paper: [5].



undirected, integer weights

After popping S

last_min = 0

bucket A : $d=1$

bucket B : $d=4$

$msdb(1, 0)=1; msdb(4, 0)=3$

Advance, settle A, relax

last_min = 1

bucket A settled

bucket B : $4 \rightarrow 3$

bucket B* stale

Advance, settle B

last_min = 3

bucket B settled

result $d=\{S:0, A:1, B:3\}$

stale B skipped on pop*

- Buckets are keyed by $msdb(d(v), last_min)$ – the most-significant differing bit between a tentative label and the current minimum.
- Advancing `last_min` re-bins live entries; lazy-stale entries (B*) are skipped on pop. Word-RAM gives $O(m + n)$; CPython does not.

DMMSY (Centerpiece)

Setting

Directed SSSP with non-negative real weights; labels use only comparison and addition.

Barrier

Dijkstra-style methods still pay for global priority ordering, giving the sparse-graph comparison baseline $O(m + n \log n)$.

DMMSY Claim

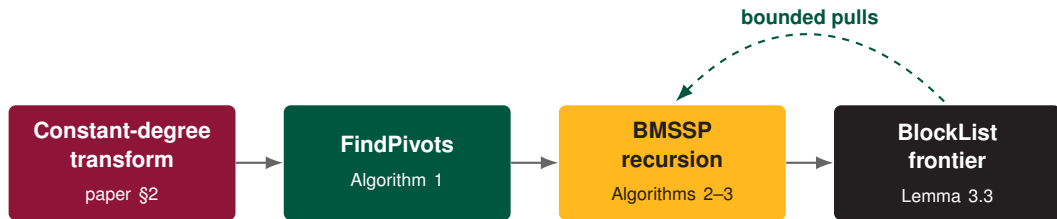
Duan–Mao–Mao–Shu–Yin break the barrier deterministically:

$$O\left(m \log^{2/3} n\right).$$

Portfolio Role

Centerpiece algorithm.
Source paper: [6].

Implementation:
`src/sssp/dmmsy/`.



- **Transform:** reduce high-degree vertices so work charges uniformly per arc.
- **FindPivots:** shrink the frontier to useful recursion roots.
- **BMSSP + BlockList:** solve bounded subproblems without globally sorting every label.

Contract

$\text{BMSSP}(G, \ell, B, S, d, k, t)$ solves labels below boundary B from source set S , then returns a smaller boundary B' and settled set U .

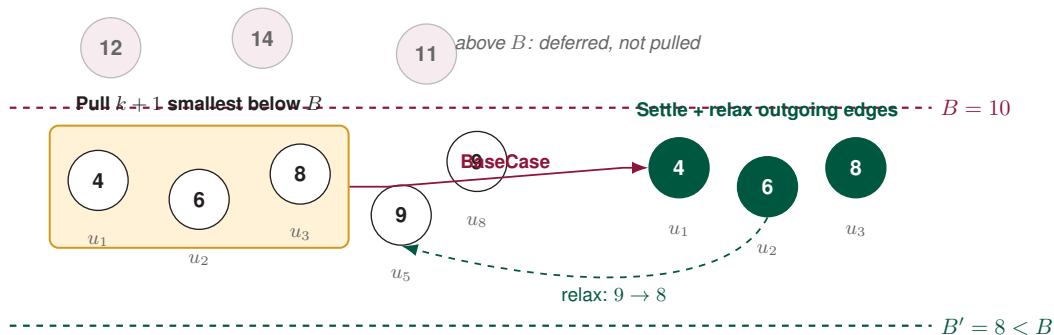
Level 0

BASECASE: bounded Dijkstra from one source, stopping after $k + 1$ discoveries or boundary exhaustion.

Level > 0

- 1 FINDPIVOTS shrinks S to pivots P .
- 2 BlockList pulls bounded batches below B .
- 3 Recursive calls settle batches and relax outgoing edges.

Key idea: local bounded batches replace one global sorted frontier.



- Pull only the $k + 1$ smallest tentative labels below B – no global sort.
- Settle, relax, return a tighter boundary $B' < B$; deferred labels stay above B .

Paper Parameterization

$$k = \left\lfloor (\log_2 n)^{1/3} \right\rfloor$$

$$t = \left\lfloor (\log_2 n)^{2/3} \right\rfloor, \quad \ell_{\max} = \left\lceil \frac{\log_2 n}{t} \right\rceil$$

Balances pivot work, recursion depth, and block-list operations.

Asymptotic Result

Deterministic directed SSSP:

$$O\left(m \log^{2/3} n\right) \text{ time}$$

Space stays $O(n + m)$.

Python Implementation Boundary

CPython recursion overhead is real. Default large numeric benchmark runs use the documented shortcut; `Weight` inputs and explicit parameters exercise recursive BMSSP.

Experimental Evaluation

Dataset

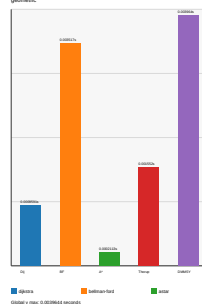
Seed 2026; 12 benchmark cases; geometric and Barabasi–Albert families at $n \in \{64, 256, 1024\}$ with sparse/dense profiles.

Timing

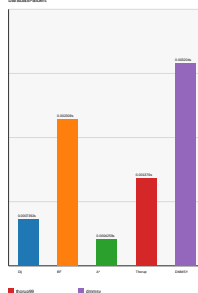
60 CSV rows; setup and file IO excluded; one recorded repeat per case-algorithm pair; Dijkstra is the normalization baseline.

Runtime by Algorithm and Graph Class

Bar height = median runtime across selected sizedensity rows.
geometric



barabasi-albert



Median runtime ratio by size bucket

Algorithm	Small $n = 64$	Medium $n = 256$	Large $n = 1024$
Dijkstra	1.00×	1.00×	1.00×
Bellman–Ford	1.92×	3.48×	3.87×
A*	0.80×	0.29×	0.50×
Thorup (1999)	1.84×	1.61×	1.36×
DMMSY (2025)	11.28×	5.87×	1.02×

- A* benefits from an informative geometric heuristic; Bellman–Ford pays for repeated edge sweeps.
- Thorup stays above Dijkstra because CPython is not the word-RAM model.
- DMMSY passes the large-size runtime sanity check: max DMMSY/Dijkstra ratio is 1.093× for $n \geq 1000$.

Notable Artifacts

- Github repository: <https://github.com/zcalifornia-ph/sssp-modern>
- CSV: `bench/results/sssp-benchmark-seed-2026.csv`
- Figure: `bench/figures/runtime-by-algorithm.{pdf,png}`
- Tests: 147-test suite passed after benchmark plot generation; the runtime check records max DMMSY/Dijkstra ratio $1.093\times$ for $n \geq 1000$.
- Report anchors: Table `tab:results-by-class` and Figure `fig:runtime-by-algorithm`.

Final Words

The portfolio centers the algorithmic result and implementation structure, not a CPython timing victory: correctness and comparison-addition fidelity are validated separately from the fast-path benchmark policy.

Reflection and Author Contributions

What The Comparison Teaches

No SSSP algorithm dominates every setting; the right choice depends on weight signs, direction, integer assumptions, heuristic quality, and runtime model.

Honest DMMSY Framing

The centerpiece is the deterministic $O(m \log^{2/3} n)$ comparison-addition bound. CPython does not show a recursive BMSSP wall-clock win; large numeric runs use the documented shortcut to keep the runtime check passing.

AI Assistance Disclosure

AI tools assisted with drafting, tests, supporting files, and prose. The source papers, validation evidence, and final judgments remain the human authors' responsibility.

Zildjian E. California

- Repository foundation, root documentation, graph/weight/IO/generator base, Dijkstra, A*, DMMSY, benchmarks, report, and deck authorship.
- DMMSY centerpiece: transform, block list, pivots, BMSSP, driver, written rationale, and presentation opening/centerpiece.
- Current commit history: authored the foundation, Dijkstra/A*, DMMSY stack, benchmark files, report manuscript, and deck scaffold.

Rey Marvin C. Rizal

- Bellman–Ford implementation with negative-cycle reporting; assigned contribution completed.
- Thorup 1999 runtime-model work and hierarchical-bucket SSSP contribution; assigned contribution completed.
- Presentation ownership: Bellman–Ford, Thorup, and experimental-results delivery; joint Q&A with California.

Contribution split follows the GitHub repository commit history and the closed issues and pull requests.

Questions?

Zildjian E. California github.com/zcalifornia-ph

Rey Marvin C. Rizal github.com/marverickdev

<https://github.com/zcalifornia-ph/sssp-modern>



University of the Philippines

The five source papers cited above are listed below; the Beamer class [7] and the UP Visual Identity Guidebook [8] are credited for the deck infrastructure.

- [1] **E. W. Dijkstra**. “A Note on Two Problems in Connexion with Graphs”. In: *Numerische Mathematik* 1.1 (1959), pp. 269–271. DOI: [10.1007/BF01386390](https://doi.org/10.1007/BF01386390).
- [2] **Richard Bellman**. “On a Routing Problem”. In: *Quarterly of Applied Mathematics* 16.1 (1958), pp. 87–90. DOI: [10.1090/qam/102435](https://doi.org/10.1090/qam/102435).
- [3] **L. R. Ford**. “Network Flow Theory”. In: P-923 (1956). Also available at <https://www.rand.org/pubs/papers/P923.html>. URL: <https://apps.dtic.mil/sti/tr/pdf/AD0422842.pdf>.
- [4] **Peter E. Hart, Nils J. Nilsson, and Bertram Raphael**. “A Formal Basis for the Heuristic Determination of Minimum Cost Paths”. In: *IEEE Transactions on Systems Science and Cybernetics* 4.2 (1968), pp. 100–107. DOI: [10.1109/TSSC.1968.300136](https://doi.org/10.1109/TSSC.1968.300136).

- [5] **Mikkel Thorup**. “Undirected Single-Source Shortest Paths with Positive Integer Weights in Linear Time”. In: *Journal of the ACM* 46.3 (1999), pp. 362–394. DOI: [10.1145/316542.316548](https://doi.org/10.1145/316542.316548).
- [6] **Ran Duan et al.** “Breaking the Sorting Barrier for Directed Single-Source Shortest Paths”. In: (2025), pp. 36–44. DOI: [10.1145/3717823.3718179](https://doi.org/10.1145/3717823.3718179).
- [7] **Till Tantau**. *The Beamer Class User Guide*. 2004.
- [8] **University of the Philippines**. *UP Visual Identity Guide 2017*. Accessed on April 2, 2026. URL: <https://up.edu.ph/wp-content/uploads/2017/03/VIG-layout-Jan-2017B-compressed.pdf>.